C# Ders - 14 Ekim 2026 Ders Notları

1. Diziler (Arrays)

Dizi, **aynı veri tipinden** birden fazla değeri bellekte ardışık (contiguous) olarak saklayan sabit boyutlu bir veri yapısıdır. C#'ta mevcut olan tüm veri tipleriyle (`int`, `string`, `double`, `char`, `bool` vb.) dizi oluşturulabilir.

A. Dizi Tanımlama ve İlklendirme (Initialization)

1. **Boyut Belirterek Tanımlama:**C#

```
// VeriTipi[] diziAdi = new VeriTipi[ElemanSayisi];
int[] numbers = new int[5]; // 5 elemanlı int dizisi (Varsayılan değerleri: 0)
string[] fruits = new string[5]; // 5 elemanlı string dizisi (Varsayılan değerleri: n
ull)
```

2. **Doğrudan Elemanları Vererek Tanımlama:**C#

```
int[] numbers2 = { 1, 2, 3, 4, 5 };
double[] prices = { 49.99, 99.99, 109.99, 150, 89.99 };
```

3. **Tanımlanıp Henüz Bellekte Alan Açılmamış Dizi:**C#

```
int[] numbers3 = null; // Bellekte adresi yok, kullanılmaya çalışılırsa NullReference
Exception verir.
```

B. İndeks Mantığı ve Eleman Erişimi

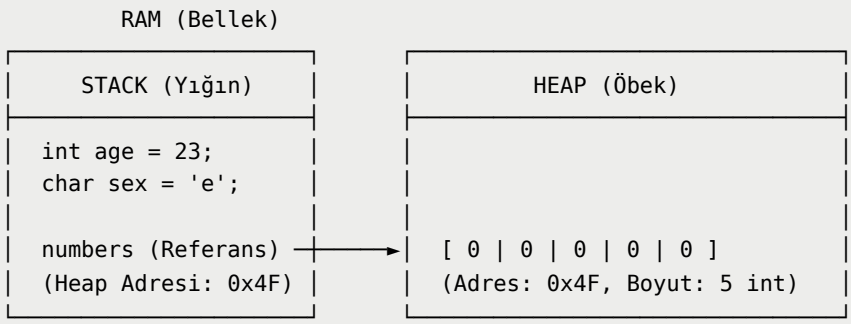
- C#'ta dizi indeksleri `0` **'dan başlar**.
- NN elemanlı bir dizinin son eleman indeksi **N−1N−1** olur.
- Dizinin eleman sayısına `dizi.Length` özelliği ile erişilir.

C#

```
string[] fruits = new string[5];
fruits[0] = "Elma";  // İlk eleman (Index 0)
fruits[1] = "Kiraz";
fruits[2] = "Muz";
fruits[3] = "Çilek";
fruits[4] = "Kivi";  // Son eleman (Index 4)
// fruits[5] = "Armut"; // HATA! IndexOutOfRangeException (Dizi sınırları aşıldı)
```

2. Derinlemesine Bellek Mimarisi: Stack vs. Heap

Kodundaki `int[] numbers = new int[5];` satırı, belleğin iki farklı alanını aynı anda kullanır. Bir C# uygulamasının RAM kullanımı temel olarak iki ana bölgeye ayrılır:



A. Stack (Yığın)

- **Çalışma Prensibi:** LIFO (*Last In, First Out* - Son giren ilk çıkar).
- **Neler Saklanır?** Değer tipleri (`int`, `double`, `float`, `char`, `bool`, `struct`) ve Heap'teki nesneleri işaret eden **referans adresleri (pointer'lar)**.
- **Yönetim:** Otomatik olarak işletim sistemi/CPU tarafından yönetilir. Bir metot veya kod bloğu (`{}`) bittiğinde, o bloğa ait değişkenler anında bellekten düşer.
- **Hız:** Çok hızlıdır; bellek tahsisi (allocation) sadece pointer'ı kaydırmaktan ibarettir.
- **Boyut:** Sabit ve sınırlıdır (genellikle 1 MB civarı). Çok derin iç içe çağrılarda veya sonsuz döngülerde taşarsa `StackOverflowException` hatası alınır.

B. Heap (Öbek)

- **Çalışma Prensibi:** Dinamik ve dağınık bellek alanı.
- **Neler Saklanır?** Referans tipleri (`string`, diziler `[]`, `class`, nesneler). `new` anahtar kelimesi ile üretilen her şey Heap'te yer açar.
- **Yönetim:** Bellek temizliği **Garbage Collector (Çöp Toplayıcı)** tarafından arka planda yürütülür. Artık hiçbir değişken tarafından işaret edilmeyen nesneler GC tarafından taranır ve bellek iade edilir.
- **Hız:** Stack'e göre daha yavaştır; çünkü RAM üzerinde boş alan aranması, tahsis edilmesi ve daha sonra Garbage Collector tarafından temizlenmesi ek maliyet getirir.
- **Boyut:** Bilgisayarın boş RAM'i kadar genişleyebilir.

C. Dizi Üzerinden Karşılaştırma

C#

```
int[] numbers = new int[5];
```

1. `numbers` isimli işaretçi değişken **Stack** üzerinde saklanır.
2. `new int[5]` ifadesi ile **Heap** üzerinde 5 adet `int` sığacak kadar (yaklaşık 5×4 byte=20 byte5×4 byte=20 byte) alan açılır.
3. Heap'teki bu alanın başlangıç bellek adresi (örneğin `0x004FFA12`), Stack'teki `numbers` değişkenine atanır.

3. Kod Analizi: Manav Kasa Hesabı ve Kritik Mantık Hatası

Kodundaki manav örneğinde mantıksal bir hata mevcuttu:

C#

```
// HATALI KULLANIM:
double totalTutar = buying[0] * prices[0];
totalTutar = buying[1] * prices[1]; // Önceki tutar ezildi!
totalTutar = buying[2] * prices[2]; // Önceki tutar ezildi!
```

```
totalTutar = buying[3] * prices[3]; // Önceki tutar ezildi!
totalTutar = buying[4] * prices[4]; // Sadece 4. meyvenin tutarı kaldı!
```

- **Hata Nedeni:** Her satırda `=` (atama) operatörü kullanıldığı için değişkenin önceki değeri kaybolur ve sonuçta sadece son meyvenin fiyatı yazdırılır.
- **Çözüm (Bileşik Atama `+=`):**

C#

```
double totalTutar = 0;
totalTutar += buying[0] * prices[0];
totalTutar += buying[1] * prices[1];
totalTutar += buying[2] * prices[2];
totalTutar += buying[3] * prices[3];
totalTutar += buying[4] * prices[4];
```

Bu şekilde tek tek 5 kere yazmak yerine bir sonraki adımda döngüler devreye girer.

4. Döngüler: `for` Döngüsü

Döngüler, belirli bir kod bloğunu belirli bir koşul sağlandığı sürece tekrar tekrar çalıştırmak için kullanılır.

`for` Döngüsü Anatomisi

C#

```
for (Sayac_Baslangic; Kosul; Artis_Miktari)
{
    // Koşul sağlandığı sürece çalışacak kodlar
}
```

C#

```
for (int i = 1; i <= 20; i++)
{
    Console.WriteLine("Dön Baba Dönelim " + i);
}
```

1. `int i = 1;` → → Döngü başlamadan önce sadece bir kez çalışır.
2. `i <= 20;` → → Her turdan önce kontrol edilir. Koşul `true` ise içeri girilir, `false` ise döngü biter.
3. `i++` → → Gövdedeki kodlar çalıştırıldıktan sonra sayacı artırır/azaltır.

5. Algoritmik Çözümler ve Kod İncelemeleri

Örnek 1: Adım Sayısını ve Mod Şartını Belirleme

1 ile 100 arasında 4'e tam bölünen sayıları ekrana basma:

C#

```
for (int i = 1; i <= 100; i++)
{
    if (i % 4 == 0)
    {
        Console.WriteLine(i);
    }
}
```

Örnek 2: 10 ile 1000 Arasındaki Palindrom Sayıları Bulma

Palindrom Sayı: Düzden ve tersten okunuşu aynı olan sayılardır (Örn: `11`, `121`, `454`, `999`).

Kodundaki matematiksel basamak ters çevirme algoritması:

C#

```
for (int i = 10; i < 1000; i++)
{
    int temp = i;     // Sayının orijinalini bozmamak için kopyasını alıyoruz
    int tersi = 0;    // Tersini oluşturacağımız değişken

    // Sayının basamaklarını mod 10 ve bölü 10 ile ters çevirme:
    tersi += temp % 10;
    temp /= 10;

    if (temp > 0)
    {
        tersi = tersi * 10 + temp % 10;
    }
```

```
    temp /= 10;

    if (temp > 0)
    {
        tersi = tersi * 10 + temp % 10;
    }

    // Orijinal sayı ile tersi aynı mı?
    if (tersi == i)
    {
        Console.WriteLine(i);
    }
}
```

> **Döngülü Genel Çözüm (Her basamak sayısı için):**
>
> Basamak sayısı kaç olursa olsun çalışan standart matematiksel yaklaşım şöyledir:
>
> ```
> int temp = sayi;
> int tersi = 0;
> while (temp > 0)
> {
>     tersi = (tersi * 10) + (temp % 10);
>     temp /= 10;
> }
> ```

Örnek 3: String Bir İfadeyi Tersten Yazdırma (String Reverse)

C#'ta `string` ifadeler arka planda birer `char[]` **(karakter dizisi)** gibi davranırlar. Dolayısıyla metnin her karakterine `yazi[i]` şeklinde indeksle erişilebilir.

C#

```
string yazi = "adanada";
string newYazi = string.Empty;

// Son karakterin indeksi: yazi.Length - 1
// İlk karakterin indeksi: 0
for (int i = yazi.Length - 1; i >= 0; i--)
{
    newYazi += yazi[i]; // Sondan başa doğru ekliyoruz
}

Console.WriteLine(newYazi); // "adanada"
```

```
if (yazi == newYazi)
{
    Console.WriteLine("Bu kelime bir palindromdur!");
}
```

6. Diziler ile `for` Döngüsünün Birleştirilmesi

Dersin başında manuel olarak 5 satırda yapılan manav hesabı, `for` döngüsü ve diziler birleştirildiğinde dinamik hale gelir:

C#

```
string[] fruits = { "Elma", "Kiraz", "Muz", "Çilek", "Kivi" };
double[] prices = { 49.99, 99.99, 109.99, 150.0, 89.99 };
double[] buying = new double[fruits.Length];
double totalTutar = 0;

// Girdileri döngü ile alma
for (int i = 0; i < fruits.Length; i++)
{
    Console.Write($"{fruits[i]} (Birim Fiyat: {prices[i]:c2}) - Kaç kilo alacaksınız? ");
    double.TryParse(Console.ReadLine(), out buying[i]);

    // Anlık olarak toplam tutara ekleme
    totalTutar += buying[i] * prices[i];
}

Console.WriteLine("-----------------------------------------");
Console.WriteLine($"Toplam Ödemeniz Gereken Tutar: {totalTutar:c2}");
```

Özet Karşılaştırma Tablosu

Kavram	Bellek Yeri	Temel Kullanım Amacı	Örnek
Değer Tipi	Stack	Tek bir basit veri saklamak	`int x = 5;`, `char c = 'A';`
Referans Tipi	Heap (Adresi Stack'te)	Kompleks, boyutlu veya dinamik veri saklamak	`string s = "Metin";`, `int[] arr = new int[5];`
`for` **Döngüsü**	CPU Yürütmesi	Adım sayısı veya sınırı baştan belli tekrarlar	`for (int i = 0; i < arr.Length; i++)`

Kavram	Bellek Yeri	Temel Kullanım Amacı	Örnek
`+=` **Operatörü**	RAM İşlemi	Kümülatif toplam tutma ve metin birleştirme	`toplam += eleman;`, `metin += "a";`